# Chuẩn hóa Jenkins cho một web-app

Công ty A đang mở rộng thương mại điện tử đa kênh. Mỗi dự án dùng pipeline khác nhau, build lâu và deploy dễ lỗi.

## Phạm vi thí điểm

Tôi chọn một web-app để chuẩn hóa trước. Giữ Jenkins, dùng Jenkinsfile chung cho logic pipeline và tách cấu hình theo dự án.

- Chạy test và review code trước khi merge.
- Dùng lại đúng image đã qua test khi release.
- Đo thời gian build và lỗi deploy trước khi mở rộng sang các ứng dụng khác.




Controller chạy trên hạ tầng ổn định; agents có thể tạo rồi xóa theo job.

## Backup và restore Controller

Backup hằng ngày dữ liệu Jenkins, cấu hình và credentials ra ngoài máy. Bảo vệ riêng khóa giải mã; định kỳ thử restore sang máy khác.

# Hai phương án chạy Jenkins agents

Controller chỉ điều phối job. Chọn hạ tầng chạy agents theo tài nguyên sẵn có và khả năng vận hành của đội ngũ.

## Docker agents trên Spot VM

**Spot VM 1** | **Spot VM 2**

↓ VM còn tài nguyên tạo agent

**Container agent**

Build · test

Job kết thúc thì xóa container; build artifacts lưu bên ngoài agent.

Mỗi VM chạy Docker. Một VM bị thu hồi thì VM còn lại nhận job mới hoặc chạy lại build bị gián đoạn. Script bên ngoài kiểm tra mỗi 10 phút, yêu cầu bật hoặc tạo lại VM.

Cả hai VM có thể bị thu hồi; không bảo đảm cấp lại ngay.

## Kubernetes khi đã có cluster

**Kubernetes plugin** tạo → **Pod agent**

↓ Pod thiếu tài nguyên để schedule

**Karpenter**

Cấp / thu gọn node

Karpenter yêu cầu cloud cấp node khi Pod thiếu tài nguyên.

Tôi chọn phương án này khi đã có cluster trên cloud hỗ trợ Karpenter và đội vận hành phù hợp. Job xong thì xóa Pod; tải giảm thì thu gọn node đủ điều kiện.

Khả năng cấp node vẫn phụ thuộc tài nguyên của nhà cung cấp.

# Tách cấu hình, secrets và quyền theo môi trường

Các dự án dùng chung logic pipeline nhưng tách cấu hình và quyền truy cập theo dự án, môi trường.

**Version template** Lưu Jenkinsfile template trong Git, review và test trước khi release. Mỗi dự án pin một version; không tự nhận thay đổi mới.

## Cấu hình qua Folder Properties

Lưu biến dùng chung trong Jenkins folder, đọc qua `withFolderProperties`. Không lưu mật khẩu hoặc access key ở đây.

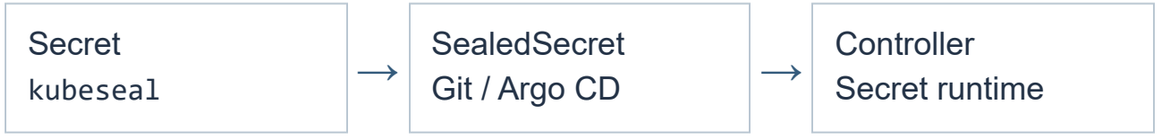
## Credentials cho pipeline

Lưu trong Jenkins Credentials, dùng `withCredentials` ở bước cần thiết. Không in secrets, kể cả khi có log masking.

## Secrets cho ứng dụng ở runtime

Cloud có Key Vault hoặc Secrets Manager. Với Kubernetes on-premises, tôi dùng Sealed Secrets: chỉ commit bản mã vào Git, không commit Secret gốc.

## Cách kubeseal hoạt động

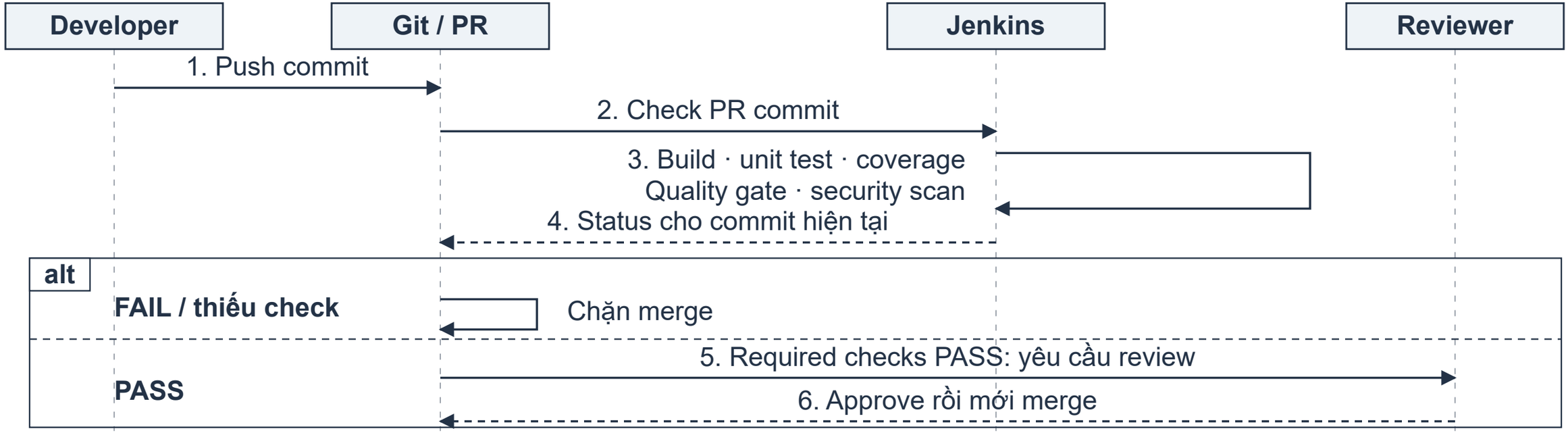



kubeseal mã hóa bằng public key. Argo CD apply bản mã; controller dùng private key tạo Secret. Backup private key, phân quyền Secret; key rotation không đổi app credentials.

**RBAC:** tách quyền và secrets giữa dev, staging, production; chỉ người có vai trò phù hợp được phê duyệt hoặc deploy production.

# Chặn lỗi trước khi merge PR

Jenkins kiểm tra commit hiện tại của PR; branch protection chặn merge nếu thiếu check hoặc approval.




Required checks PASS và reviewer approval đều phải áp dụng cho commit hiện tại.

## Required checks

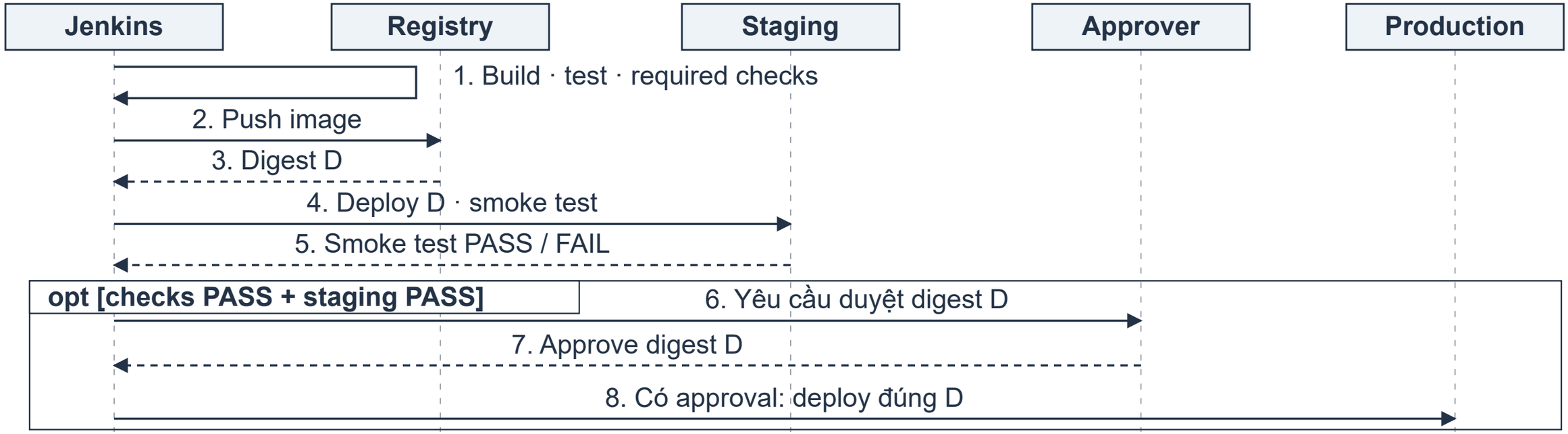
Build, unit test, code coverage, SonarQube quality gate và Trivy security scan theo policy dự án.

## Commit mới phải chạy lại

Push commit mới phải chạy lại checks và review; kết quả commit cũ không cho phép merge.

# Release đúng image đã qua test

Sau merge, Jenkins build một image; chỉ deploy production khi staging PASS và có approval.




Production dùng đúng digest đã qua staging; checks FAIL hoặc thiếu approval thì dừng release.

## Kiểm tra và rollback

Rolling update có readiness probe và timeout. Deploy lỗi thì rollback về digest ổn định, kiểm tra lại và cảnh báo.

## Giữ tương thích dữ liệu

Rollback ứng dụng không khôi phục dữ liệu. Database schema mới phải cho phép bản cũ tiếp tục chạy.

# Tăng agents theo hàng đợi, đặt lock đúng đích deploy

Tăng agents khi job phải chờ lâu. Build có thể chạy song song; deploy cùng ứng dụng và môi trường phải dùng chung lock.

## Scale theo tải thực tế

**Hàng đợi**
Số job · thời gian chờ

xét thêm →

**Agents**
Giới hạn CPU / RAM

**VM** Chạy nhiều container agents trong giới hạn CPU và RAM. Thiếu tài nguyên thì thêm VM; dùng labels để chọn đúng nhóm agent.

**Kubernetes** Đặt requests/limits cho Pod. Karpenter cấp node khi thiếu tài nguyên; giới hạn tổng tài nguyên và tách build khỏi production workloads.

## Chạy song song các bước độc lập

web-app và warehouse-app có thể build đồng thời. Trong một pipeline, lint và unit test cũng có thể chạy song song.

Lint

Unit test

Tách workspace nếu hai nhánh có thể ghi đè file của nhau.

## Lock theo ứng dụng + môi trường

web-app
production · lượt A

nhả lock →

web-app
production · lượt B

Ứng dụng hoặc môi trường khác vẫn được deploy song song.

# Tối ưu Dockerfile và nơi lưu cache

Cache chỉ có tác dụng khi builder truy cập được cache cũ. Phương án này chưa có shared cache giữa các VM hoặc node.

## Cài dependencies trước, copy code sau

```
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build
```

Thứ tự các bước trong Dockerfile của web-app Node.js.

Giữ nguyên các layer trước và hai file package thì Docker có thể dùng lại layer `npm ci`. Chỉ đổi code không buộc cài lại dependencies.

## Cache không tự đi theo agent




Đổi host không tự chuyển local cache sang host mới.

Cùng Docker daemon, cache có thể còn sau khi xóa container agent. Builder riêng hoặc storage bị xóa thì phải build lại khi không có cache.

## Reuse image khác với reuse cache

Image trong registry là build artifact để deploy; dùng lại image không tự chia sẻ build cache giữa agents.

Đo thời gian build cùng host và khác host; thêm remote cache khi thời gian tiết kiệm đủ bù chi phí lưu trữ và truyền tải.

# Theo dõi lỗi và đo hiệu quả deploy

Dùng Jenkins để theo dõi agents, hàng đợi, thời gian chạy và log. Chưa cần dựng dashboard riêng.

## Thông báo qua Teams hoặc email

Khi pipeline kết thúc, gửi status của build và deploy riêng. Kèm ứng dụng, môi trường, commit/digest, bước lỗi và link job; không gửi secrets.

## Probe Jenkins từ máy độc lập




Probe mỗi phút, timeout 10 giây; kiểm tra HTTP 200 và nội dung Jenkins.

Lỗi 3 lần liên tiếp thì gửi email; phục hồi thì báo lại. Probe chỉ xác nhận truy cập được Jenkins, không xác nhận build chạy được.

## Ba metrics theo tuần/tháng

### Deployment frequency

Số lượt deploy production thành công của từng ứng dụng trong kỳ.

### Build success rate

Build PASS / (PASS + FAIL). Bỏ lượt bị hủy; không tính kết quả deploy vào tỷ lệ build.

### MTTR

Thời gian trung bình từ lúc phát hiện sự cố đến khi dịch vụ phục hồi; chỉ tính các sự cố đã phục hồi.

Không có sự cố thì MTTR không có mẫu, không mặc định bằng 0.

So sánh xu hướng qua từng kỳ để chọn bước cải tiến tiếp theo.